

SPRINT 1

JalenOS — Sprint 1: Core API Reliability

Sprint Goal: Make Priority Management a reliable, validated, automatically tested backend capability that is ready to move from temporary in-memory storage to a database in Sprint 2.

Duration: 2 weeks

Committed stories: PRI-04, PRI-05, PRI-06, ENG-01A

The sprint is successful even if we build **zero new visible features**. We're turning the prototype into a trustworthy API.

PRI-04 — Partially Update a Priority

As a JalenOS user, I want to update selected fields of an existing priority so that I can change its state without recreating it.

Size: M

Priority: Must

Status: In Progress

Acceptance Criteria

- PATCH accepts one or multiple editable fields.
- Fields omitted from the request remain unchanged.
- `priority_id` cannot be modified through the request body.
- Updated record remains a complete `Priority` object.
- Existing `priority_id` remains unchanged.
- Valid update returns the complete updated `Priority`.
- Missing priority returns 404.
- Invalid update data is rejected by validation.
- Updated record appears correctly through subsequent GET requests.

Engineering Tasks

- Finish `PriorityUpdate` model.
- Determine editable vs immutable fields.
- Extract only fields actually supplied in PATCH.
- Merge submitted fields with existing `Priority`.
- Replace existing item without changing list order.
- Remove unnecessary model/dictionary conversions.

- Confirm response matches `response_model`.
- Manually test through `/docs`.

You were basically at the merge/replacement portion when you stopped.

Research

Start: [FastAPI — Partial Updates](#)

Deeper: [Pydantic — Model Serialization](#)

PRI-05 — Delete a Priority

As a JalenOS user, I want to remove obsolete priorities so that JalenOS contains only relevant work.

Size: S

Priority: Must

Acceptance Criteria

- Existing priority can be deleted by ID.
- Deleted priority disappears from GET all.
- GET by its former ID returns 404.
- Deleting nonexistent ID returns 404.
- DELETE response contract is deliberate and consistent.
- If using 204 No Content, response contains no body.

Engineering Tasks

- Decide between 200 + response versus 204 + no response.
- Correct existing endpoint accordingly.
- Test successful deletion.
- Test nonexistent ID.
- Verify subsequent GET behavior.

This should be a relatively quick story because most of it already exists.

Research: [FastAPI — Response Status Codes](#)

PRI-06 — Validate Priority Data

This is probably the most educational story this sprint.

As a JalenOS user, I want invalid Priority data rejected so that JalenOS maintains consistent and trustworthy information.

Size: M

Priority: Must

Dependency: Stable CRUD contract

Acceptance Criteria

We need to define your actual **business rules**, rather than validating something merely because Pydantic can.

For Alpha, I'd propose:

Title

- Required when creating.
- Cannot be empty/whitespace.
- Reasonable maximum length.

Category

- Must belong to a supported JalenOS domain.
- No arbitrary category strings.

Something like your eventual domain set:

Career / Projects / Investments / Health / Learning / Personal

We can refine this before implementation.

Importance

- Restricted to defined levels.
- No arbitrary strings.

For example:

Low / Medium / High / Critical

Status

- Restricted to defined states.
- Valid default exists during creation.

Action

- Decide what `action` actually represents.
- Validate accordingly.

This deserves attention because earlier your code set:

```
action = "Pending"
```

and I suspect we're mixing **action** and **status** conceptually.

We should fix the data model now rather than carry bad semantics into PostgreSQL.

Link

- Optional.
- When supplied, must represent an appropriate URL.

ID

- One ID strategy.
- New records use UUIDs.
- Clients cannot choose IDs.
- Malformed route IDs are handled predictably.

Engineering Tasks

- Write Priority business rules before coding.
- Refine `Priority`.
- Refine `PriorityCreate`.
- Refine `PriorityUpdate`.
- Research Pydantic field constraints.
- Research enums/literal values.
- Add field validation.
- Test intentionally invalid POST requests.
- Test intentionally invalid PATCH requests.
- Inspect generated OpenAPI `/docs` schema.

Start: [FastAPI — Body Fields](#)

Deeper: [Pydantic — Fields](#)

Custom business rules: [Pydantic — Validators](#)

ENG-01A — Automated Priority API Tests

This is where your development process changes significantly.

As a JalenOS developer, I want automated tests for Priority API behavior so that future changes cannot silently break the core system.

Size: M/L because this is new territory

Priority: Must

Dependency: PRI-04-06

Acceptance Criteria

Your test suite verifies at minimum:

- GET all succeeds.
- GET existing Priority succeeds.
- GET nonexistent Priority → 404.
- Valid POST → 201.
- Invalid POST is rejected.
- POST creates retrievable Priority.
- PATCH modifies submitted fields.
- PATCH preserves omitted fields.
- PATCH nonexistent Priority → 404.
- DELETE existing Priority succeeds.
- DELETE nonexistent Priority → 404.
- Deleted Priority cannot subsequently be retrieved.

And importantly:

- Tests can be executed with **one command**.
- Tests do not depend on manually opening Swagger.
- Test results clearly identify failures.

Engineering Tasks

Don't jump straight into writing ten tests.

- Learn pytest fundamentals.
- Learn FastAPI `TestClient`.
- Create test structure.
- Write **one GET test**.
- Run it successfully.

- Add GET failure test.
- Add POST tests.
- Add PATCH tests.
- Add DELETE tests.
- Ensure tests don't contaminate each other's data.

That last item will become especially important once the database arrives.

Start: [FastAPI — Testing](#)

Testing framework: [pytest — Get Started](#)

Advanced/production direction: [FastAPI — Testing Dependencies with Overrides](#)

Dependency overrides will become much more relevant when Sprint 2 introduces database dependencies.

Sprint Definition of Done

This is different from acceptance criteria.

A **story's acceptance criteria** tell you whether that feature behaves correctly.

Your **Definition of Done** tells you what "finished engineering work" means.

For Sprint 1, a story isn't Done until:

- Acceptance criteria satisfied.
- Successful path tested.
- Relevant failure paths tested.
- API response matches declared response model.
- Appropriate HTTP status codes are used.
- No known traceback/error occurs during expected usage.
- Code saved and committed to Git.
- Any important architectural decision documented.
- Story manually verified before moving to Done.

Once ENG-01A exists, automated tests become part of the DoD for future backend stories.